Whole-Line Column Generation for Multi-Line Human–Robot Collaborative Selective Disassembly With Cumulative Fatigue

Author Names and Affiliations to Be Confirmed

***Abstract*—Multi-line disassembly requires coordination of limited human and robot resources for profitable material recovery. This work considers selective task execution, three operation modes, AND precedence relationships, and cumulative station fatigue. A mixed-integer linear model maximizes net profit under cycle-time, recovery, and shared-resource constraints. A whole-line column generation framework integrates feasible line plans, two-phase resource coordination, restricted integer recovery, and demand-driven resource-profile bounds. Computational experiments establish integer optimality for six synthetic instances with 80 or 120 tasks in 19.36–182.69 seconds under a 600-second protocol, whereas direct compact-model optimization remains unproven on these instances; additional resource-constrained cases confirm the benefit, and a difficult two-line case identifies its boundary.**

***Index Terms*—Human–robot collaboration, disassembly line balancing, column generation, cumulative fatigue, resource allocation.**

I. Introduction

DISASSEMBLY separates end-of-life products into components and materials that can re-enter production. Profitable recovery requires more than removing every accessible part. A low-value operation can consume scarce station time, whereas an apparently unprofitable predecessor can provide access to a valuable component. The disassembly line balancing problem (DLBP) therefore connects task selection, precedence, station assignment, and recovery economics. Guo *et al.* review the modeling and optimization of disassembly sequences [1]. Özceylan *et al.* survey line-balancing formulations and solution approaches [2]. These perspectives motivate treating selection and allocation as a coordinated decision.

Selective disassembly makes this coordination particularly important. Only part of a product may justify recovery, but required operations and minimum recovery targets still restrict the selected subset. Güler *et al.* organize developments in partial disassembly line balancing [3]. Laili *et al.* compare models and algorithms for multi-objective disassembly balancing [4]. He *et al.* examine uncertainty and links to supply-chain decisions [5]. These reviews show that problem variants differ in both feasible operations and performance criteria. A comparison of solution methods consequently needs a fixed mathematical model and identical input data. The present work uses deterministic task data and one profit objective so that formulation and certification effects remain identifiable.

Human–robot collaboration adds another layer of operational choice. Human execution can offer flexibility, robot execution can provide a different time–cost combination, and joint execution uses both resources. Xu *et al.* study collaborative line balancing with a safe strategy [6]. Parsa and Saadat investigate collaborative planning for end-of-life product disassembly [7]. Xiao and Huang review collaboration in uncertain and dynamic disassembly [8]. Yin *et al.* formulate multi-man–robot line balancing with mathematical programming and a genetic Jaya algorithm [9]. These studies support explicit mode decisions, while their safety, workforce, and uncertainty assumptions need not coincide with those of a deterministic shared-resource model.

Layout also changes which decisions interact. Wu *et al.* consider two-sided balancing with human–robot interaction constraints [10], and Chu and Chen study partial disassembly in a two-sided cooperative setting [11]. Al-fandi and Al-masarwah examine mixed-model two-sided disassembly [12]. Huang *et al.* introduce activation decisions in a collaborative U-shaped line [13]. Fang *et al.* address position-constrained collaborative planning [14]. Battaïa and Dolgui review hybrid line-balancing structures [15], while Boysen *et al.* review broader assembly-balancing developments [16]. These contributions indicate why layout assumptions require explicit treatment. Here a physical circular conveyor has a fixed entry and an ordered, single-pass task sequence. Its geometric shape does not introduce another product lap or a cyclic precedence decision.

Profit-oriented collaboration also interacts with operation-specific restrictions. Guo *et al.* study hybrid partial destructive disassembly with noise and component accessibility considerations [17]. Zhu *et al.* consider remanufacturing demand and hazardous components [18]. Goksoy Kalaycilar *et al.* address hazardous task failures [19]. Zhang *et al.* maximize collaborative disassembly profit with an improved grey wolf optimizer [20]. Wu *et al.* investigate stochastic processing and carbon-emission objectives [21], and Guo *et al.* consider stochastic operation times [22]. The present model retains task-level recovery value and mode-dependent execution cost. It does not import hazard, failure, carbon, or stochastic constraints from these distinct variants.

Human workload requires a similarly precise definition. Guo *et al.* consider a human fatigue index in U-shaped disassembly balancing [23]. Otto and Scholl incorporate ergonomic risks into assembly balancing [24]. Xu and Han explicitly include worker rest time in a two-sided disassembly model [25]. Zhou *et al.* consider worker fatigue in multi-factory remanufacturing [26]. This work limits the cumulative fatigue load at each human-equipped station over one representative cycle. It in-

cludes no recovery period and makes no claim about fatigue across an entire shift. This bounded interpretation keeps the ergonomic assumption consistent with the implemented additive-load constraints.

Multiple lines create a further coordination problem even when every individual plan is feasible. Their combined human and robot deployments can exceed the available pools. Guo *et al.* study resource sharing in multiproduct human–robot collaborative hybrid lines [27]. Gu *et al.* optimize multi-factory remanufacturing with shared transportation [28]. The shared quantities here are fixed station-level human and robot deployments, rather than transport movements. All task, precedence, recovery, time, and fatigue requirements remain within a line. Only two aggregate resource rows connect different lines. This structure suggests a decomposition that preserves complete local plans.

A compact formulation in mixed-integer linear programming (MILP) provides a direct baseline and an independent definition of feasibility. Its linear programming (LP) relaxation, however, can combine fractional task assignments inside stations. A whole-line formulation instead expresses each line as a convex combination of complete feasible integer plans. Dantzig and Wolfe establish the underlying decomposition principle [29], and Lübbecke and Desrosiers explain pricing and dual bounds in column generation (CG) [30]. Local integer convexification can strengthen the relaxation, but the corresponding line-pricing problem is itself a MILP. The computational benefit therefore depends on the balance between local pricing difficulty and global coordination, rather than total task count alone.

Integer recovery presents a separate issue. An optimal restricted master LP neither guarantees an integer combination of its columns nor proves the original integer optimum. Barnhart *et al.* describe branch-and-price as a framework for integrating pricing into integer search [31]. Pessoa *et al.* demonstrate generic exact-solver development for routing-related problems [32]. These works establish useful methodological distinctions, rather than a claim that the present implementation performs a branch-and-price tree search. For the two shared resource counts considered here, resource-profile value functions permit an alternative integer coordination mechanism using dynamic programming.

Algorithmic improvements also require evidence about where time and proof gaps arise. du Merle *et al.* study stabilized CG [33], and Maher and Rönnberg investigate pricing schemes that promote high-quality integer solutions [34]. Their methods illustrate that pricing behavior and integer solution quality are distinct design concerns. This work uses exact line pricing, verified restricted integer recovery, and demand-driven queries of unresolved resource profiles. It reports valid upper bounds alongside feasible profits and distinguishes LP convergence from integer bound closure. A retained difficult case with few large lines identifies a regime where decomposition loses its advantage.

The choice of a column also affects practical feasibility. With station columns, individually legal configurations can omit required tasks, duplicate a task, or disagree on cross-station precedence when combined. A whole-line column resolves these interactions before entering the master. The remaining resource conflict can be described by two integer counts, without replacing any original task constraint by a penalty. This design trades a harder local solve for a smaller coordination problem. It motivates reporting both pricing effort and integer certification, and testing configurations with different line-block sizes. It also makes the limits of the approach visible: a compact model can remain preferable when local blocks become too difficult or shared resources impose little coordination.

This work makes the following new contributions:

1) A whole-line reformulation is developed for selective multi-line collaborative disassembly with cumulative station fatigue. Complete line plans preserve all local compatibility requirements, and bidirectional mappings establish integer equivalence with the compact model. The relaxation hierarchy explains the source and limits of formulation strength.
2) A solution framework combines two-phase CG, independently checked integer recovery, and demand-driven resource-profile bounds. Explicit pricing states and repaired dual bounds distinguish feasible solutions, complete LP proofs, and original integer optimality. Profile coordination can close an integer gap without asserting LP convergence.
3) A reproducible comparison uses frozen paired synthetic inputs and reports profits, valid bounds, proof states, pricing rounds, and execution times. Integer certification is faster on the tested four- and six-line configurations. Resource-ample and large-block cases delimit this empirical finding.

Section II defines the problem and compact model. Section III presents the proposed algorithm and its bounds. Section IV reports the computational evidence. Section V states the conclusions and remaining questions.

II. Problem Description and Mathematical Model

A. System Definition and Assumptions

Several lines process different end-of-life products while drawing humans and robots from shared pools. Fig. 1 illustrates a serviceable electric fan unit and its recovery operations. It is a conceptual example for explaining task relationships, not an industrial measurement or an experimental input. Each line has an AND precedence graph. An executed successor requires every direct predecessor. Optional tasks can remain unexecuted; required tasks must be executed.

The illustrative tasks remove the front guard (T_1), detach the impeller (T_2), open the rear cover (T_3), disconnect the controller (T_4), extract the motor unit (T_5), separate the rotor (T_6), recover the stator (T_7), and recover the power lead (T_8). These labels connect the component tree to the access graph.

Fig. 2 shows a physical circular layout. A fixed entry cuts each circle into ordered stations. A product visits these stations once and exits; only empty carriers return. Consequently, station indices have the same precedence meaning as an ordered line. The model contains no multi-lap product routing, geometric travel time, or cyclic task sequence. The figure

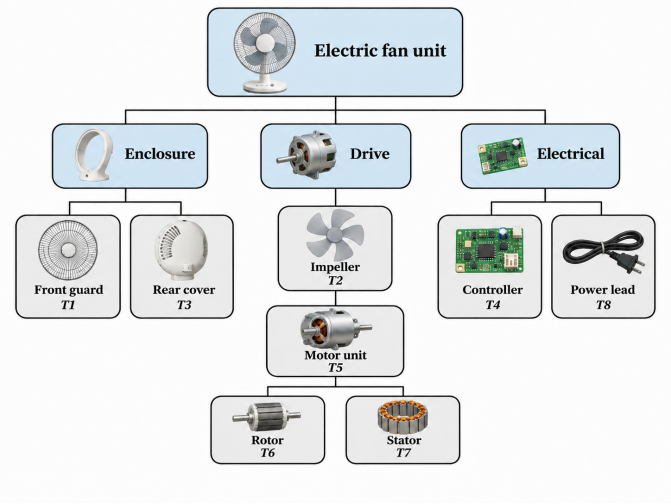


Fig. 1. Illustrative fan-unit product structure and recovery operations. The component tree indicates containment, not precedence; the task graph in Fig. 3 defines access dependencies.

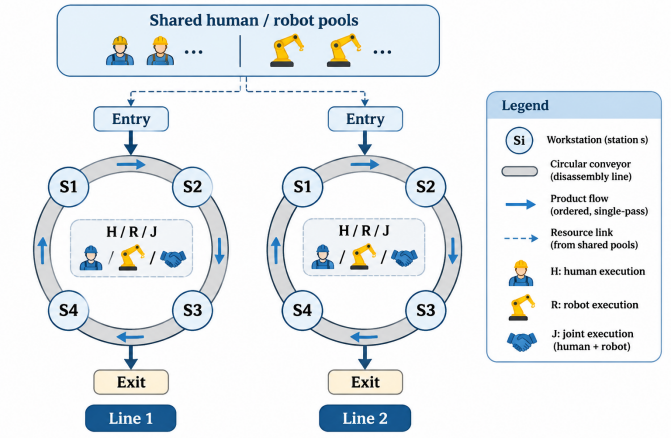


Fig. 2. Two illustrative circular lines with fixed entry, ordered single-pass processing, and shared planning pools. Dashed return arcs carry empty carriers only. Pool links indicate deployment limits, not movement during operation.

illustrates the adopted layout without changing the underlying assignment constraints.

Modes H, R, and J denote human, robot, and joint execution. Mode J requires one human and one robot; its duration enters station time once. Tasks within a station execute sequentially. A selected task receives exactly one allowed mode at exactly one station. Fig. 3 distinguishes AND access dependencies from the OR choice among execution modes. OR does not denote mutually exclusive product-disassembly routes. A predecessor can share a station with its successor; a topological order gives a feasible within-station sequence.

Fatigue starts at zero in a representative cycle. Each human-involving task contributes a nonnegative additive load. There is no within-cycle recovery or accumulation across successive cycles. Robot-only modes have zero fatigue load. Resource pools limit the total number of stations with human and robot deployment. This planning model does not describe resource movement. Candidate stations can have different costs, and consecutive activation is not required. There are no precedence

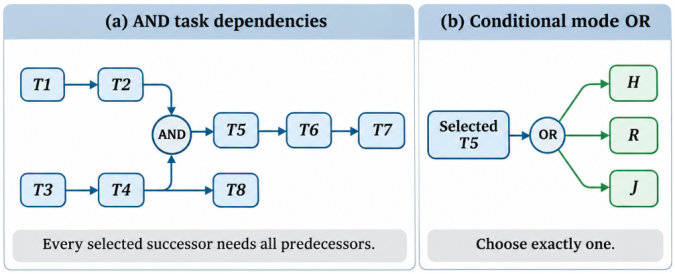


Fig. 3. Illustrative AND dependencies and conditional execution-mode OR. Both T_2 and T_4 precede T_5. If T_5 is selected, exactly one allowed mode is chosen; omitting an optional task selects none. The diagram introduces no route-selection OR constraint.

arcs between lines.

B. Notation

1) Sets and indices:

- $\mathcal{L}$: Set of disassembly lines, indexed by l.
- $\mathcal{I}_l$: Tasks on line l, indexed by i, j; $n_l = |\mathcal{I}_l|$.
- $\mathcal{I}_l^{\text{req}}$: Subset of required tasks on line l.
- $\mathcal{S}_l$: Ordered candidate stations $\{1, \ldots, k_l\}$, indexed by s; k denotes a station prefix endpoint.
- $\mathcal{M}_{li}$: Allowed modes for task i, a subset of $\{H, R, J\}$, indexed by m.
- $\mathcal{E}_l$: Direct AND arcs (i, j), where i precedes j.

2) Parameters:

- v_{li}, w_{li}: Recovery value and recovery weight of task i.
- c_{li}^m, t_{li}^m: Direct execution cost and processing duration in mode m.
- β_{li}^m: Fatigue coefficient; $d_{li}^m = \beta_{li}^m t_{li}^m$ is the corresponding load.
- $c_{ls}^0, c_{ls}^H, c_{ls}^R$: Station activation, human deployment, and robot deployment costs.
- t_l^c: Cycle-time limit on line l.
- $\bar{f}_l, b_l$: Fatigue threshold and additive budget, with $0 \le \bar{f}_l < 1$ and $b_l = -\ln(1 - \bar{f}_l)$.
- q_l: Minimum recovered weight on line l.
- $h^{\max}, r^{\max}$: Shared human and robot capacities.
- a_m^H, a_m^R: Resource indicators, equal to $(1, 0)$ for H, $(0, 1)$ for R, and $(1, 1)$ for J.

3) Decision variables:

- y_{li}: One if task i is selected, and zero otherwise.
- x_{lism}: One if task i executes at station s in mode m.
- u_{ls}: One if station s is active.
- h_{ls}, r_{ls}: Human and robot deployment indicators at station s.

C. Compact Formulation

Every sum over m uses the task's allowed set $\mathcal{M}_{li}$. Unless a row states otherwise, task rows apply to all l, i and station rows to all l, s. Net profit is maximized:

$$\max \; z = \sum_{l,i} v_{li} y_{li} - \sum_{l,i,s,m} c_{li}^m x_{lism} - \sum_{l,s} \left(c_{ls}^0 u_{ls} + c_{ls}^H h_{ls} + c_{ls}^R r_{ls} \right). \quad (1)$$

$$\sum_{s\in\mathcal{S}_l}\sum_{m\in\mathcal{M}_{li}} x_{lism} = y_{li} \qquad \forall l,i, \tag{2}$$

$$y_{li} = 1 \qquad \forall l,\ i\in\mathcal{I}_l^{\mathrm{req}}. \tag{3}$$

For every l, $(i,j)\in\mathcal{E}_l$, and $k\in\mathcal{S}_l$, AND precedence requires

$$\sum_{s=1}^{k}\sum_{m\in\mathcal{M}_{lj}} x_{ljsm} \le \sum_{s=1}^{k}\sum_{m\in\mathcal{M}_{li}} x_{lism}. \tag{4}$$

$$\sum_{i,m} t_{li}^m x_{lism} \le t_l^c u_{ls}, \tag{5}$$

$$u_{ls} \le \sum_{i,m} x_{lism} \le n_l u_{ls}, \tag{6}$$

$$h_{ls} \le \sum_{i,m} a_m^H x_{lism} \le n_l h_{ls}, \tag{7}$$

$$r_{ls} \le \sum_{i,m} a_m^R x_{lism} \le n_l r_{ls}, \tag{8}$$

$$\sum_{i,m} d_{li}^m x_{lism} \le b_l h_{ls}. \tag{9}$$

$$\sum_{l,s} h_{ls} \le h^{\max}, \tag{10}$$

$$\sum_{l,s} r_{ls} \le r^{\max}, \tag{11}$$

$$\sum_{i\in\mathcal{I}_l} w_{li} y_{li} \ge q_l \qquad \forall l, \tag{12}$$

$$x_{lism}, y_{li}, u_{ls}, h_{ls}, r_{ls} \in \{0,1\}. \tag{13}$$

Objective (1) subtracts execution, station, human, and robot costs from task recovery value.

Constraint (2) indicates that a selected task receives exactly one station–mode assignment; an omitted task receives none. It enforces the conditional mode OR in Fig. 3.

Constraint (3) indicates that every required task is selected.

Constraint (4) indicates that each successor requires its predecessor at the same or an earlier station. The final prefix enforces predecessor selection. Applying the row to every incoming arc gives AND precedence.

Constraint (5) indicates that total station processing time does not exceed the line cycle limit when the station is active.

Constraint (6) indicates that a station is active if and only if it contains at least one task. Thus an empty station incurs no activation cost.

Constraint (7) indicates that a human is deployed if and only if at least one assigned mode requires a human.

Constraint (8) indicates the corresponding if-and-only-if requirement for robot deployment. Together these rows charge both resources for mode J and for a station mixing H and R tasks.

Constraint (9) indicates that cumulative human load stays within its additive budget. Equivalently, the station fatigue level $1-\exp(-\sum_{i,m} d_{li}^m x_{lism})$ does not exceed $\bar{f}_l$ at a human-equipped station.

Constraint (10) indicates that total human deployment across all lines does not exceed the shared capacity.

Constraint (11) indicates the analogous limit for robots.

Constraint (12) indicates that every line meets its minimum recovered weight.

Constraint (13) indicates the binary domains of task, assignment, activation, and deployment decisions. These definitions retain the mathematical feasible set of the implemented compact model.

III. Proposed Algorithm

A. Whole-Line Plans

Let $\mathcal{X}_l$ contain all binary solutions of line l that satisfy (2)–(9) and (12). The shared limits in (10) and (11) remain outside this set. A pricing model can use the redundant bounds $\sum_s h_{ls} \le k_l$ and $\sum_s r_{ls} \le k_l$. These bounds do not allocate global resources to a line.

A column $p\in\mathcal{P}_l$ represents one complete plan in $\mathcal{X}_l$. It records the selected tasks, their stations and modes, and the associated station configurations. Define

$$h_{lp} = \sum_s h_{ls}^p, \qquad r_{lp} = \sum_s r_{ls}^p, \tag{14}$$

$$c_{lp} = \sum_i v_{li} y_{li}^p - \sum_{i,s,m} c_{li}^m x_{lism}^p - \sum_s (c_{ls}^0 u_{ls}^p + c_{ls}^H h_{ls}^p + c_{ls}^R r_{ls}^p). \tag{15}$$

An empty station is allowed within a plan. A completely empty line plan exists only if the required-task and recovery conditions allow it. Columns with the same selected task set can still differ in station assignment or mode. Deduplication therefore uses the complete assignment pattern.

Fig. 4 shows the solution workflow. A station-level column can be locally feasible but difficult to combine across stations. A whole-line column already satisfies task uniqueness, required-task selection, all precedence prefixes, and line recovery. Integer combination then requires coordination of the two resource totals. The two-phase CG loop supplies LP coordination, while restricted integer recovery and profile queries address the remaining integer gap.

B. Master Problem and Formulation Strength

Binary λ_{lp} selects one plan for line l. The complete master problem is

$$\max \quad \sum_l \sum_{p\in\mathcal{P}_l} c_{lp}\lambda_{lp}, \tag{16}$$

$$\text{s.t.} \quad \sum_{p\in\mathcal{P}_l} \lambda_{lp} = 1 \qquad \forall l, \tag{17}$$

$$\sum_{l,p} h_{lp}\lambda_{lp} \le h^{\max}, \tag{18}$$

$$\sum_{l,p} r_{lp}\lambda_{lp} \le r^{\max}, \tag{19}$$

$$\lambda_{lp} \in \{0,1\}. \tag{20}$$

Column profit already includes all station and resource costs.

Proposition 1. *The complete integer master and the compact MILP have the same feasible objective values.*

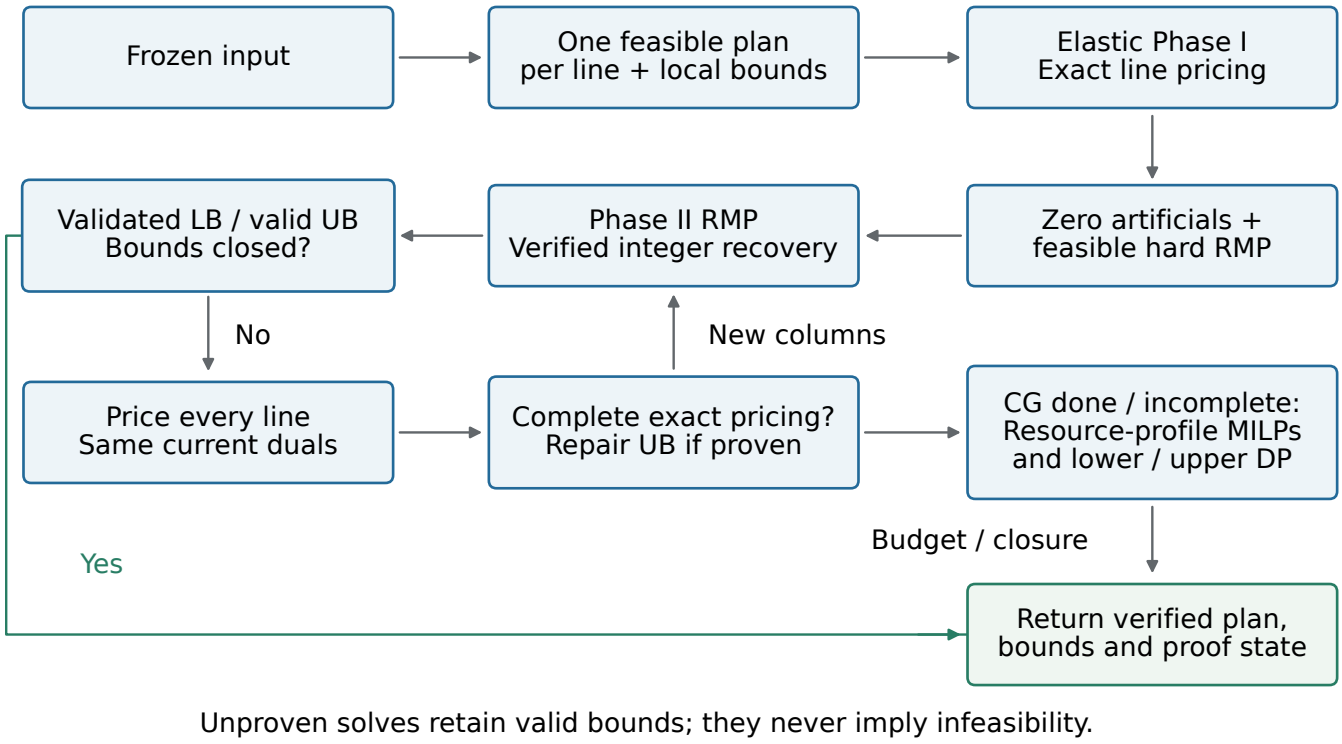


Fig. 4. Proposed framework. CG coordinates complete line plans; verified integer recovery and resource-profile bounds provide separate integer proof paths. Only complete exact pricing repairs the CG dual bound. A budget stop returns the actual proof state and remaining gap.

Proof. Extract the binary variables of each line from a feasible compact solution. They form a member of $\mathcal{X}_l$ and hence a column. Selecting these columns preserves the resource totals and every objective term. Conversely, select one column per line from a feasible integer master solution. Copy its task, station, and mode variables to the compact model. Local feasibility follows from $\mathcal{X}_l$. Shared feasibility follows from (18) and (19). The two-way configuration logic fixes activation and deployment, including empty stations. The reconstructed profit is unchanged. □

The complete linear programming (LP) relaxation replaces (20) by $\lambda_{lp} \geq 0$. Let $z^{\mathrm{LP}}_{\mathrm{line}}$, $z^{\mathrm{LP}}_{\mathrm{station}}$, and $z^{\mathrm{LP}}_{\mathrm{compact}}$ denote the complete line, complete station, and compact LP bounds. Then

$$z^{\mathrm{IP}} \leq z^{\mathrm{LP}}_{\mathrm{line}} \leq z^{\mathrm{LP}}_{\mathrm{station}} \leq z^{\mathrm{LP}}_{\mathrm{compact}}. \tag{21}$$

Every line-plan mixture induces station-plan mixtures with identical weights and objective. Station-plan mixtures in turn induce feasible compact fractional assignments. This establishes the inclusions in (21). The supplementary file gives the detailed mapping. These comparisons concern complete column sets, not arbitrary restricted pools.

C. Restricted Master and Exact Pricing

CG maintains a finite pool $\overline{\mathcal{P}}_l \subseteq \mathcal{P}_l$. Its restricted master problem (RMP) uses the LP version of (16)–(19) over this pool. The dual variables are $\sigma_l \in \mathbb{R}$ for (17) and $\pi_H, \pi_R \geq 0$ for the resource rows. The complete dual is

$$\min \quad D = \sum_l \sigma_l + h^{\max}\pi_H + r^{\max}\pi_R, \tag{22}$$

$$\text{s.t.} \quad \sigma_l + \pi_H h_{lp} + \pi_R r_{lp} \geq c_{lp} \qquad \forall l, p. \tag{23}$$

The reduced cost of a column is

$$\bar{c}_{lp} = c_{lp} - \pi_H h_{lp} - \pi_R r_{lp} - \sigma_l. \tag{24}$$

Pricing solves one MILP per line:

$$\rho_l = \max_{p \in \mathcal{P}_l} \{c_{lp} - \pi_H h_{lp} - \pi_R r_{lp} - \sigma_l\}. \tag{25}$$

The objective increases the effective deployment costs by π_H and π_R. All line constraints remain unchanged. Task coverage and precedence do not have separate master duals in this formulation. They belong to the pricing domain.

A new legal column enters the pool if its independently recomputed reduced cost exceeds ε_{rc}. All lines use the same RMP dual solution in a round. The implementation reuses one native CPLEX model per line. It updates every objective coefficient and submits the previous full integer solution as a warm start. Separate line sessions prevent state contamination.

D. Initialization and Phase I

A short economic MILP first seeks a feasible plan on each line. A constant-objective solve provides a fallback. A proven empty local domain implies global infeasibility. A time limit without a plan does not establish emptiness.

Local plans can conflict in their combined resource demands. Phase I keeps each convexity equation hard and adds nonnegative elastic variables:

$$\begin{aligned} \max \quad & -a_H - a_R, && (26)\\ \text{s.t.} \quad & \sum_p \lambda_{lp} = 1 && \forall l,\\ & \sum_{l,p} h_{lp}\lambda_{lp} - a_H \le h^{\max},\\ & \sum_{l,p} r_{lp}\lambda_{lp} - a_R \le r^{\max},\\ & \lambda_{lp}, a_H, a_R \ge 0. \end{aligned}$$

One local plan per line makes this elastic RMP feasible. Real columns have zero objective coefficients. The dual satisfies $0 \le \pi_H, \pi_R \le 1$. Phase I pricing maximizes $-\pi_H h_{lp} - \pi_R r_{lp} - \sigma_l$ over $\mathcal{P}_l$.

The complete Phase I optimum is zero exactly when the complete line LP is feasible. A feasible line LP admits zero artificials. Conversely, zero artificials leave all real master constraints satisfied. This equivalence concerns LP feasibility. It does not establish integer feasibility. After the artificial sum reaches its tolerance, the implementation resolves the hard Phase II RMP before proceeding.

E. Valid Bounds and Integer Recovery

At an optimal RMP solution, repair each convexity dual by $\sigma'_l = \sigma_l + \max(0, \rho_l)$. Complete exact pricing then gives

$$z^{\mathrm{LP}}_{\mathrm{line}} \le u^{CG} = D + \sum_l \max(0, \rho_l). \quad (27)$$

The repaired variables satisfy every complete-dual inequality. This proves the bound by weak duality. When every $\rho_l \le \varepsilon_{rc}$, the residual LP gap can accumulate across the lines. It is not automatically bounded by one pricing tolerance.

An incomplete or time-limited pricing round does not certify (27). The implementation still attempts the remaining lines under the same duals. It then stops that CG loop if any pricing maximum remains unproven. Existing valid bounds remain available. A negative complete Phase I repaired bound proves LP infeasibility.

The restricted integer master provides candidate integer plans. A data-level checker and the original compact constraints verify each plan before its profit becomes a lower bound (LB). Restricted integer optimality alone does not imply global integer optimality. Independently, local economic upper bounds $b^e_l \ge \max_{p\in\mathcal{P}_l} c_{lp}$ give $z^{\mathrm{IP}} \le z^{\mathrm{LP}}_{\mathrm{line}} \le \sum_l b^e_l$. The implementation also reuses reliable native bounds from full-domain, zero-resource-price economic pricing. It restores the convexity constant before using these bounds. It does not substitute an incumbent for a maximization upper bound (UB).

F. Demand-Driven Resource Profiles

When the current LB and UB remain separated, define

$$v_l(a,b) = \max\{c_{lp} : p \in \mathcal{P}_l,\ h_{lp} = a,\ r_{lp} = b\}. \quad (28)$$

The integer counts satisfy $a \le \min(k_l, h^{\max})$ and $b \le \min(k_l, r^{\max})$. An infeasible profile has value $-\infty$. A profile with no completed solve remains unresolved. Plans with identical resource totals have identical coupling coefficients. Only the most profitable plan of such a profile is needed for integer coordination.

Let $\underline{v}_l$ contain validated profile incumbents and $\overline{v}_l$ contain valid profile upper bounds. Unknown upper bounds remain $+\infty$. Prior complete pricing gives

$$\overline{v}_l(a,b) \le \sigma_l + \max(0, \rho_l) + \pi_H a + \pi_R b. \quad (29)$$

The right-hand side is a valid candidate bound; the stored value takes the minimum over valid sources. For a profile table v, dynamic programming (DP) uses

$$f_l(A,B) = \max_{a\le A, b\le B}\{f_{l-1}(A-a, B-b) + v_l(a,b)\}, \quad (30)$$

with $f_0(A,B) = 0$. The lower table yields a feasible integer combination. The upper table bounds every feasible integer combination. Its bound can be tighter than the complete line LP bound. Closing this integer bound does not certify LP convergence.

The procedure queries an unresolved profile on an upper-DP maximizing path. It chooses the least-attempted profile and uses deterministic line and profile ordering to break ties. A query fixes exact resource counts in a local economic MILP. Its incumbent improves the lower table. Its valid native bound improves the upper table. Its proven infeasibility removes that profile. Query budgets increase geometrically with repeated attempts. The procedure recomputes both DP tables until the budget ends or the bounds close.

Algorithm 1 summarizes the implemented framework. Let the best verified integer profit be ℓ and the best valid integer upper bound be u. Define

$$g_{abs} = u - \ell, \qquad g_{rel} = \frac{\max(0, u-\ell)}{\max(1, |\ell|)}. \quad (31)$$

Missing bounds produce an unknown gap. Integer optimality requires

$$u - \ell \le \max\{\varepsilon_{abs}, \varepsilon_{rel}\max(1, |\ell|)\}. \quad (32)$$

A bound below the verified LB beyond the consistency tolerance triggers a numerical error. All proofs use solver floating-point contracts. They are not exact rational certificates. The reported implementation combines root CG and profile DP. Its experiments do not measure a branch-and-price search tree.

IV. Experimental Studies

A. Test Instances and Evaluation Protocol

The experiments use synthetic inputs from a seeded generator. They do not represent measured industrial products. The main set contains three seeds, 32–34, at two configurations: 80 tasks on four lines and 120 tasks on six lines. Each line has 20 tasks and eight candidate stations. Both methods read the same frozen input bytes. The task graph is layered and contains only within-line forward arcs. Required tasks occupy the first 20% of each line's task list. Recovery requires at least 35% of the total task weight, rounded upward.

Algorithm 1 Whole-Line CG With Integer Coordination

Input: Line data, shared capacities, time budget, tolerances.
Output: Verified plan, bounds, and proof state.

Generate one feasible plan per line by local MILP.
Record reliable local economic bounds.
Check independent-plan compatibility and bound closure.
Solve elastic Phase I by CG; retain explicit pricing states.
Resolve the hard Phase II RMP after zero artificials.
while time remains and Phase II remains unresolved **do**
    Solve the RMP and verify its primal and dual solution.
    Recover and verify a restricted integer solution.
    Stop if the verified LB closes the valid UB.
    Price every line under the current duals.
    Add legal new positive-reduced-cost columns.
    Update the UB only after complete exact pricing.
    Stop CG on an incomplete pricing proof or LP convergence.
end while
Query unresolved resource profiles through upper-DP paths.
Update integer bounds and verify the selected plan.
Return the actual bound, proof state, and termination reason.

The generator samples mode availability and task economics before partitioning the task bank. It provides a structurally feasible construction for input checks. Neither solver receives this construction as an initial solution. Shared capacities in the main set equal the observed independent economic calibration counts minus one for each resource. This calibration defines a development design, not a random industrial population. The supplementary file provides the generator rules and all capacity values.

A second set retains the 120-task inputs and sets each shared capacity to the ceiling of 80% of its calibrated independent use. A retained 80-task two-line input provides a large-block boundary. A separate structure study compares two, four, and eight lines at 80 total tasks. Repartitioning retains the task bank but changes precedence graphs and recovery rows. These are different mathematical inputs. The study therefore does not isolate line count as the only causal factor.

The direct baseline solves the compact MILP with CPLEX. The decomposition also uses CPLEX inside line pricing, profile queries, and integer recovery. All measured pairs use the same machine, CPLEX 22.1.1, one thread, random seed 11, and zero requested MIP gap. The archived environment contains Python 3.10.11, PuLP 3.3.2, and Windows 10. The physical processor model is not preserved in the archived environment and requires confirmation for submission.

The main protocol allocates 600 seconds per method. The resource-constrained protocol allocates 180 seconds. Both reserve five seconds for evidence finalization. A parent process measures execution from worker startup through exit, including required result persistence. Independent offline replay is separate. The watchdog and native solver limits operate under Windows scheduling; the protocol is not a hard real-time guarantee. All listed main and robustness workers return timely, complete evidence. A time-limited CPLEX completion time is censored. It is not an observed time to optimality. No 3600-second run enters these tables.

TABLE I
SMALL COMPLETE-COLUMN CHECKS

Case	Line LP	Full integer	Profile result
Five-task reference	24.00	24.00	Optimal
Six-task reference	56.63	56.63	Optimal
Positive-gap case	6.50	2.00	Optimal
LP-feasible/IP-infeasible	4.00	Infeasible	Infeasible

B. Parameters and Performance Metrics

The framework uses a 0.5-second economic initialization per line and a 0.5-second feasibility fallback when necessary. Restricted integer recovery receives up to 0.5 seconds per call. CG receives at most 90% of the available internal budget. The pricing loop divides remaining time among the remaining lines. Initial profile queries receive up to two seconds. Repeated attempts double that allowance, subject to the remaining global time. The maximum CG record count is 100. These settings are fixed for the reported main and resource-constrained sets. They are implementation settings, not parameters inferred from a cited study.

The LP primal and dual tolerances are 10^{-8}. Pricing integrality, reduced-cost, and artificial tolerances are 10^{-7}. Original-solution verification uses 10^{-6}. Bound consistency and absolute integer closure use 10^{-5}. Relative integer closure uses 10^{-8}. The supplementary file lists all numerical roles.

The primary metrics are the validated integer LB, valid UB, gap in (31), integer proof state, and measured execution time. Additional metrics include pricing calls, generated columns, Phase I and II pricing rounds, and native solver time. A round count excludes RMP-only phase transitions and early closure records. Compact-model search nodes and CG rounds represent different operations and are not directly compared.

C. Feasibility and Integer-Certificate Checks

Small instances use an independent enumerator of task omission, station assignment, and allowed mode. The enumerator checks task selection, AND precedence, time, fatigue, deployment, and recovery directly from data. An independently assembled complete-column LP and integer model provide reference values. Archived checks also compare CPLEX and CBC on these small cases.

Table I confirms the need to distinguish LP and integer conclusions. The positive-gap case requires integer coordination despite exact CG convergence. The LP-feasible/IP-infeasible case contains no integer witness. Resource-profile analysis proves its integer infeasibility. These small archived tests establish implementation consistency within their enumerated scope. They do not replace larger-instance performance evaluation.

TABLE II
PROFIT AND BOUNDS UNDER THE 600-SECOND PROTOCOL

Instance N/L/seed	Compact CPLEX				Whole-line framework		
	LB	UB	Gap (%)	Time (s)	LB	UB	Time (s)
80/4/32	583.64	618.26	5.93	595.68	583.64	583.64	19.36
80/4/33	675.21	719.63	6.58	595.73	675.21	675.21	24.60
80/4/34	667.64	720.51	7.92	595.84	667.64	667.64	68.68
120/6/32	954.82	1007.66	5.53	595.75	963.44	963.44	36.51
120/6/33	1009.46	1066.51	5.65	595.78	1012.19	1012.19	105.57
120/6/34	1041.21	1097.26	5.38	595.80	1041.21	1041.21	182.69

The whole-line framework proves integer optimality in every listed row. Compact CPLEX remains time-limited and unproven. Times include required evidence persistence. Rounded equal bounds indicate closure within the recorded tolerances.

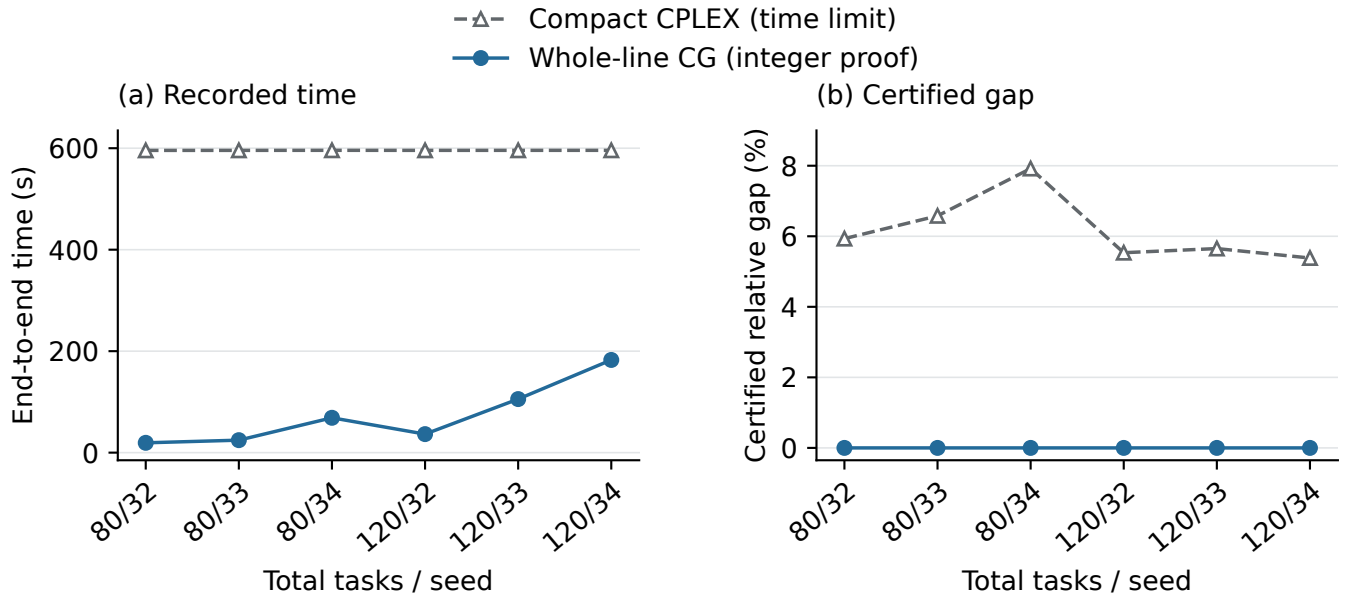


Fig. 5. All six paired main inputs, ordered by total tasks and seed: (a) recorded end-to-end time; (b) final certified relative gap. Open triangles indicate time-limited, unproven compact results; filled circles indicate completed decomposition integer proofs. Compact times are censored observations, not times to optimality. Lines connect distinct inputs only as visual guides; they are not convergence trajectories.

D. Main Comparison With Compact CPLEX

Table II shows an integer-certification advantage on all six main inputs. The framework establishes optimality in 19.36–182.69 seconds. Compact CPLEX remains unproven within the 600-second protocol. On the three 80-task inputs, both methods reach the same final integer profit. Their difference concerns proof completion. On two 120-task inputs, the framework also returns a higher final integer profit. On seed 34, Compact CPLEX reaches the same profit but retains a positive bound gap.

Fig. 5 presents the same comparisons without converting cutoffs into exact speedup ratios. These results support efficient certification on the tested multi-line configurations. They do not establish that the framework first discovers an optimal-profit plan earlier. Comparable first-optimum timestamps are not available for Compact CPLEX.

E. Resource Coordination and a Large-Block Boundary

Table III extends the comparison to stronger resource competition. The framework proves the three 120-task optima under the 180-second protocol. Compact CPLEX remains unproven and returns lower profits. Seeds 32 and 33 close through verified integer plans and complete CG bounds. Seed 34 requires 31 demand-driven profile queries after an incomplete CG pricing round. Its final certificate concerns the original integer optimum. It does not assert complete line-LP convergence.

The two-line case in Table III provides a contrary result. Two lines with 40 tasks each produce difficult local MILPs. The decomposition returns both a lower profit and a looser UB than Compact CPLEX. Its CG pricing consumes about 156 seconds, and subsequent profile queries do not close the gap. Thus total task count alone does not predict the benefit of decomposition.

The separate 80-task structure study provides further context. A resource-constrained eight-line input reaches the same proven optimum under both methods. The framework completes in about 2.91 seconds; Compact CPLEX completes in about 128.09 seconds. A resource-ample eight-line input instead favors Compact CPLEX, at about 1.24 versus

TABLE III
RESOURCE-CONSTRAINED AND LARGE-BLOCK CASES UNDER THE 180-SECOND PROTOCOL

Instance N/L/seed	$h^{\max}/r^{\max}$	Compact CPLEX			Whole-line framework			
		LB	UB	Time (s)	LB	UB	Time (s)	Proven
80/2/31	6/7	693.08	733.55	175.64	569.53	738.73	174.97	No
120/6/32	10/9	911.91	1010.76	175.59	927.46	927.46	30.73	Yes
120/6/33	9/10	955.90	1069.88	175.56	971.93	971.93	61.51	Yes
120/6/34	10/10	980.67	1101.05	175.55	1009.98	1009.98	144.55	Yes

The 120-task inputs use 80% of calibrated independent resource use, rounded upward. The 80-task two-line row is the retained large-block boundary. Compact CPLEX is unproven in all four rows.

TABLE IV
PRICING ROUNDS AND NATIVE SOLVER COSTS IN THE MAIN EXPERIMENT

N/L/seed	i_I	i_{II}	PP	Cols.	PP (s)	RMP (s)
80/4/32	1	3	20	18	17.67	0.0064
80/4/33	1	4	24	22	22.79	0.0077
80/4/34	1	4	24	21	66.70	0.0073
120/6/32	1	3	30	26	34.24	0.0058
120/6/33	1	5	42	28	102.94	0.0076
120/6/34	1	4	36	27	180.31	0.0056

A pricing round includes an RMP solution and at least one pricing attempt under its duals. PP includes economic initialization. RMP-only transition and bound-closure records do not count as pricing rounds. Resource-profile query counts are zero in these six runs.

1.64 seconds. The supplementary tables retain every compact/decomposition pair in that structure study. The evidence supports a tradeoff between block difficulty and coordination, rather than a universal benefit from adding lines.

F. Iterations, Cost, and Interpretation

Table IV shows that Phase II needs only three to five pricing rounds in the main experiment. Native pricing nevertheless accounts for about 91–99% of total framework time. Each RMP uses less than 0.01 second of native solve time. The restricted integer master also remains small. These observations identify local pricing difficulty as the main cost in this regime.

The whole-line formulation can strengthen the relaxation through local integer convexification. It can also simplify integer combination by placing all within-line compatibility inside each column. The experiments are consistent with these mechanisms. They do not isolate relaxation strength from warm-start behavior or solver search-state effects. A full component ablation is still required to quantify each contribution.

The evaluation has a defined boundary. It uses a small synthetic development suite and one solver seed per measured input. Resource calibration is part of the instance construction. The suite does not establish statistical significance, industrial ergonomic validity, or performance on arbitrary large line blocks. Broader comparator algorithms, measured product cases, and an independent final test suite remain necessary before a broad state-of-the-art claim.

V. CONCLUSION

This work develops a whole-line decomposition framework for multi-line selective human–robot disassembly with cumulative fatigue. Complete line plans preserve local feasibility and expose only two shared-resource totals to the master problem. Two-phase CG supplies LP coordination and valid bounds. Restricted integer recovery and demand-driven resource profiles supply separate integer witnesses and proof paths.

The tested multi-line configurations show a certification advantage over direct compact-model optimization. The retained large-block case defines a practical limitation: difficult local MILPs can dominate the computation. The results therefore support decomposition for the tested structure, rather than unconditional superiority at larger task counts.

Further investigation concerns stronger line-pricing bounds, allocation of certification budgets, and measured-product validation. Broader independent instances and additional optimization baselines are necessary to assess how far the observed certification advantage extends beyond the tested configurations.

REFERENCES

[1] X. Guo, M. Zhou, A. Abusorrah, F. Alsokhiry, and K. Sedraoui, "Disassembly sequence planning: A survey," *IEEE/CAA Journal of Automatica Sinica*, vol. 8, no. 7, pp. 1308–1324, 2021.

[2] E. Özceylan, C. B. Kalayci, A. Güngör, and S. M. Gupta, "Disassembly line balancing problem: a review of the state of the art and future directions," *International Journal of Production Research*, vol. 57, no. 15-16, pp. 4805–4827, 2019.

[3] E. Güler, C. B. Kalayci, M. A. Ilgin, E. Özceylan, and A. Güngör, "Advances in partial disassembly line balancing: A state-of-the-art review," *Computers & Industrial Engineering*, vol. 188, p. 109898, 2024.

[4] Y. Laili, Y. Li, Y. Fang, D. T. Pham, and L. Zhang, "Model review and algorithm comparison on multi-objective disassembly line balancing," *Journal of Manufacturing Systems*, vol. 56, pp. 484–500, 2020.

[5] J. He, F. Chu, A. Dolgui, and M. F. Anjos, "Multi-objective disassembly line balancing and related supply chain management problems under uncertainty: Review and future trends," *International Journal of Production Economics*, vol. 272, p. 109257, 2024.

[6] W. Xu, J. Cui, B. Liu, J. Liu, B. Yao, and Z. Zhou, "Human-robot collaborative disassembly line balancing considering the safe strategy in remanufacturing," *Journal of Cleaner Production*, vol. 324, p. 129158, 2021.

[7] S. Parsa and M. Saadat, "Human-robot collaboration disassembly planning for end-of-life product disassembly process," *Robotics and Computer-Integrated Manufacturing*, vol. 71, p. 102170, 2021.

[8] J. Xiao and K. Huang, "A comprehensive review on human–robot collaboration remanufacturing towards uncertain and dynamic disassembly," *Manufacturing Review*, vol. 11, p. 17, 2024.

[9] T. Yin, J. Wu, J. Cao, Y. Huang, C. Li, and J. Long, "Multi-man–robot collaborative disassembly line balancing optimization via mixed-

integer programming and genetic Jaya algorithm," *Journal of Cleaner Production*, vol. 490, p. 144695, 2025.

[10] T. Wu, Z. Zhang, Y. Zhang, and Y. Zeng, "Modelling and optimisation of two-sided disassembly line balancing problem with human–robot interaction constraints," *Expert Systems with Applications*, vol. 230, p. 120589, 2023.

[11] M. Chu and W. Chen, "Human-robot cooperation two-sided partial disassembly line balancing problem," *International Journal of Production Research*, vol. 63, no. 12, pp. 4478–4503, 2025.

[12] L. Al-fandi and N. Almasarwah, "Optimising and modeling the mixed-model two-sided disassembly line balancing problem with human-robot cooperation restrictions," *International Journal of Production Research*, vol. 63, no. 12, pp. 4389–4412, 2025.

[13] Z. Huang, H. Zhang, G. Tian, M. Yang, D. Wang, and Z. Li, "Energy-efficient human-robot collaborative U-shaped disassembly line balancing problem considering turn on-off strategy: Uncertain modeling and solution method," *Journal of Manufacturing Systems*, vol. 80, pp. 38–69, 2025.

[14] Y. Fang, Z. Li, S. Wang, and X. Lu, "Multi-objective multi-fidelity optimisation for position-constrained human-robot collaborative disassembly planning," *International Journal of Production Research*, vol. 62, no. 11, pp. 3872–3889, 2024.

[15] O. Battaïa and A. Dolgui, "Hybridizations in line balancing problems: A comprehensive review on new trends and formulations," *International Journal of Production Economics*, vol. 250, p. 108673, 2022.

[16] N. Boysen, P. Schulze, and A. Scholl, "Assembly line balancing: What happened in the last fifteen years?" *European Journal of Operational Research*, vol. 301, no. 3, pp. 797–814, 2022.

[17] L. Guo, Z. Zhang, T. Wu, Y. Zhang, Y. Zeng, and X. Xie, "Green and efficient-oriented human-robot hybrid partial destructive disassembly line balancing problem from non-disassemblability of components and noise pollution," *Robotics and Computer-Integrated Manufacturing*, vol. 90, p. 102816, 2024.

[18] L. Zhu, Y. Chen, and J. Mumtaz, "Multi-objective human-robot collaborative disassembly line balancing considering components remanufacture demand and hazard characteristics," *Computers & Industrial Engineering*, vol. 197, p. 110621, 2024.

[19] E. Goksoy Kalaycilar, M. Azizoğlu, and S. Batun, "Disassembly line balancing with hazardous task failures – Model based solution approaches," *Computers & Industrial Engineering*, vol. 190, p. 110089, 2024.

[20] Z. Zhang, S. Dai, C. Li, W. Wang, J. Parron, and E. Herrera, "Human-Robot Collaborative Disassembly Profit Maximization via Improved Grey Wolf Optimizer," *International Journal of Artificial Intelligence and Green Manufacturing*, vol. 1, no. 2, pp. 69–79, 2025.

[21] T. Wu, Z. Zhang, L. Guo, H. Song, X. Xie, and S. Ren, "A hybrid evolutionary algorithm for the stochastic human–robot collaborative disassembly line balancing problem considering carbon emission optimization," *Engineering Applications of Artificial Intelligence*, vol. 135, p. 108703, 2024.

[22] X. Guo, C. Fan, M. Zhou, S. Liu, J. Wang, S. Qin, and Y. Tang, "Human–robot collaborative disassembly line balancing problem with stochastic operation time and a solution via multi-objective shuffled frog leaping algorithm," *IEEE Transactions on Automation Science and Engineering*, vol. 21, no. 3, pp. 4448–4459, 2024.

[23] X. Guo, T. Wei, J. Wang, S. Liu, S. Qin, and L. Qi, "Multiobjective U-Shaped Disassembly Line Balancing Problem Considering Human Fatigue Index and an Efficient Solution," *IEEE Transactions on Computational Social Systems*, vol. 10, no. 4, pp. 2061–2073, 2023.

[24] A. Otto and A. Scholl, "Incorporating ergonomic risks into assembly line balancing," *European Journal of Operational Research*, vol. 212, no. 2, pp. 277–286, 2011.

[25] Z. Xu and Y. Han, "Two sided disassembly line balancing problem with rest time of works: A constraint programming model and an improved NSGA II algorithm," *Expert Systems with Applications*, vol. 239, p. 122323, 2024.

[26] L. Zhou, H. Zhu, and B. Akbari, "Multi-objective optimization of multi-factory remanufacturing process considering worker fatigue," *International Journal of Artificial Intelligence and Green Manufacturing*, vol. 1, no. 2, pp. 93–107, 2025.

[27] X. Guo, F. Guo, L. Qi, J. Wang, S. Liu, S. Qin, and W. Wang, "Modeling and optimization of multiproduct human–robot collaborative hybrid disassembly line balancing with resource sharing," *IEEE Transactions on Computational Social Systems*, vol. 12, no. 5, pp. 2848–2863, 2025.

[28] J. Gu, Z. Guo, J. Wang, L. Qi, S. Qin, and S. Zhang, "Optimization of multi-factory remanufacturing processes with shared transportation resources using the ALNS algorithm," *International Journal of Artificial Intelligence and Green Manufacturing*, vol. 1, no. 1, pp. 1–13, 2025.

[29] G. B. Dantzig and P. Wolfe, "Decomposition principle for linear programs," *Operations Research*, vol. 8, no. 1, pp. 101–111, 1960.

[30] M. E. Lübbecke and J. Desrosiers, "Selected topics in column generation," *Operations Research*, vol. 53, no. 6, pp. 1007–1023, 2005.

[31] C. Barnhart, E. L. Johnson, G. L. Nemhauser, M. W. P. Savelsbergh, and P. H. Vance, "Branch-and-Price: Column Generation for Solving Huge Integer Programs," *Operations Research*, vol. 46, no. 3, pp. 316–329, 1998.

[32] A. Pessoa, R. Sadykov, E. Uchoa, and F. Vanderbeck, "A generic exact solver for vehicle routing and related problems," *Mathematical Programming*, vol. 183, no. 1-2, pp. 483–523, 2020.

[33] O. du Merle, D. Villeneuve, J. Desrosiers, and P. Hansen, "Stabilized column generation," *Discrete Mathematics*, vol. 194, no. 1-3, pp. 229–237, 1999.

[34] S. J. Maher and E. Rönnberg, "Integer programming column generation: accelerating branch-and-price using a novel pricing scheme for finding high-quality solutions in set covering, packing, and partitioning problems," *Mathematical Programming Computation*, vol. 15, no. 3, pp. 509–548, 2023.